Phase 6 — Advanced ML

ExamGuard AI — Implementation Guide

LSTM & RNN — Teaching AI to Understand TIME

What Is This?

Regular neural networks look at ONE thing at a time. Show it a photo, it tells you what is in the photo. Done.

But what if the answer depends on what happened BEFORE?

Think about reading a sentence:

"The cat sat on the ___"

You know the answer is probably "mat" or "chair" because you read the PREVIOUS words. You understood the SEQUENCE.

RNN (Recurrent Neural Network) = A neural network that has MEMORY. It reads things one by one, and remembers what it saw before.

LSTM (Long Short-Term Memory) = An improved RNN that remembers things for a LONGER time. Regular RNN forgets quickly. LSTM has a special "memory cell" that decides what to remember and what to forget.

How Does It Work? (Simple Explanation)

Regular Neural Network (No Memory)

Photo 1 → Model → "Student looking left"
Photo 2 → Model → "Student leaning"
Photo 3 → Model → "Hand moving"

Each photo analyzed SEPARATELY. No connection between them.

LSTM Network (Has Memory)

Photo 1 → LSTM → "Student looking left" (remembers this)
Photo 2 → LSTM → "Student leaning" (remembers: looked left, THEN leaned)
Photo 3 → LSTM → "Hand moving" (remembers: looked left, leaned, THEN hand moved)

LSTM says: "This SEQUENCE = cheating pattern!"

The Key Idea: Hidden State

LSTM has a "hidden state" — think of it as a sticky note that gets passed from one step to the next.

Step 1: Read frame 1 → Write on sticky note: "looked left"
Step 2: Read frame 2 + sticky note → Update sticky note: "looked left, then leaned"
Step 3: Read frame 3 + sticky note → Update sticky note: "looked left, leaned, hand moved"
Step 4: Read sticky note → CHEATING PATTERN DETECTED

The Three Gates

LSTM has three "gates" that control memory:

- 1. Forget Gate — "Should I forget old information?"
- Student went back to writing 5 minutes ago? Forget the old glance.
- 2. Input Gate — "Should I remember this new information?"
- Student looking at neighbor RIGHT NOW? Yes, remember this!
- 3. Output Gate — "What should I tell the next step?"
- Based on everything remembered, what is the current assessment?

WHY ExamGuard Needs This

The Problem With Single-Frame Detection

If you only look at ONE frame:

- Student looking left → Could be thinking, looking at clock, stretching
- Student leaning → Could be tired, adjusting seat
- Hand near face → Could be scratching nose

Each action alone is INNOCENT.

The Power of Sequences

But when you see the SEQUENCE:

Frame 1-10: Student writing normally
Frame 11-15: Looks at neighbor's desk (not the clock, not the wall — the DESK)
Frame 16-20: Leans toward neighbor
Frame 21-25: Hand moves to cover their own paper
Frame 26-30: Writes rapidly (copying?)
Frame 31-35: Returns to normal position

This sequence SCREAMS cheating! No single frame proves it, but the pattern does.

ExamGuard LSTM Pipeline

Camera captures 30 frames per second
|
v
Every 10 frames → Extract features using CNN
|
v
Feed features to LSTM (sequence of 10 feature vectors)
|
v
LSTM outputs: "Cheating probability: 87%"
|
v
Because it saw the SEQUENCE, not just one moment

Real-World Example

Think about how a human invigilator catches cheating:

They do NOT look at one frozen moment. They WATCH behavior over time:

"Hmm, that student glanced at the neighbor..."
"Now they're leaning..."
"Now they're writing fast..."
"OK, that's suspicious. I'm going to watch more closely."

LSTM does EXACTLY this — it watches over time and builds suspicion.

What You Need to Learn

Step 1: Understand Sequence Data

- Time series data (stock prices over days)
- Text data (words in a sentence)
- Video data (frames over time)
- All of these are SEQUENCES where order matters

Step 2: RNN Basics

```
# Conceptual RNN (not real code yet)
hidden_state = zeros  # Start with blank memory

for each_frame in video:
    features = CNN(each_frame)  # Extract what's in the frame
    hidden_state = RNN(features, hidden_state)  # Update memory
    prediction = classify(hidden_state)  # What does the sequence mean?
```

Step 3: Why RNN Fails (Vanishing Gradient)

- RNN forgets things that happened long ago
- After 20+ steps, early information is basically gone
- This is called the "vanishing gradient problem"

Step 4: LSTM Solves This

```
import torch
import torch.nn as nn

# Create an LSTM layer
lstm = nn.LSTM(
    input_size=256,    # Size of features from each frame
    hidden_size=128,   # Size of the memory
    num_layers=2,      # Stack 2 LSTM layers
    batch_first=True   # Input shape: (batch, sequence, features)
)

# Feed a sequence of 10 frames
# Each frame has 256 features (from CNN)
input_sequence = torch.randn(1, 10, 256)  # 1 video, 10 frames, 256 features

output, (hidden, cell) = lstm(input_sequence)
# output shape: (1, 10, 128) — prediction at each step
# hidden: final memory state
# cell: final cell state (long-term memory)
```

Step 5: Full ExamGuard LSTM Model

```
class ExamGuardLSTM(nn.Module):
  def __init__(self):
    super().__init__()
    # CNN to extract features from each frame
    self.cnn = models.resnet18(pretrained=True)
    self.cnn.fc = nn.Linear(512, 256)  # Reduce to 256 features

    # LSTM to understand the sequence
    self.lstm = nn.LSTM(
      input_size=256,
      hidden_size=128,
      num_layers=2,
      batch_first=True
    )

    # Final classification
    self.classifier = nn.Linear(128, 2)  # Normal vs Cheating

  def forward(self, video_frames):
    # video_frames shape: (batch, 10_frames, 3, 224, 224)
    batch_size, seq_len = video_frames.shape[:2]

    # Extract features from each frame
    features = []
    for t in range(seq_len):
      frame_features = self.cnn(video_frames[:, t])
      features.append(frame_features)

    features = torch.stack(features, dim=1)  # (batch, 10, 256)

    # Feed sequence to LSTM
    lstm_out, _ = self.lstm(features)

    # Use the LAST output (after seeing all frames)
    last_output = lstm_out[:, -1, :]  # (batch, 128)

    # Classify
    prediction = self.classifier(last_output)
    return prediction  # Normal or Cheating
```

Step 6: Key Concepts to Master

- Sequence length: How many frames to look at (10? 30? 60?)
- Hidden size: How much memory the LSTM has
- Num layers: Stacking LSTMs for deeper understanding
- Bidirectional: Looking at sequence forward AND backward
- Attention: Focusing on the MOST IMPORTANT frames in the sequence

Mini Project: Next Word Predictor

Build this to understand how sequences work before applying to video.

Goal

Train a model to predict the next word in a sentence.

Steps

Step 1: Prepare Data

```
text = "the cat sat on the mat the dog sat on the rug"
words = text.split()
# Create vocabulary
vocab = list(set(words))
word_to_idx = {word: i for i, word in enumerate(vocab)}
idx_to_word = {i: word for word, i in word_to_idx.items()}
```

Step 2: Create Sequences

```
# Use 3 words to predict the 4th
sequences = []
for i in range(len(words) - 3):
    input_words = [word_to_idx[w] for w in words[i:i+3]]
    target_word = word_to_idx[words[i+3]]
    sequences.append((input_words, target_word))

# Example: ["the", "cat", "sat"] → "on"
```

Step 3: Build LSTM Model

```
class NextWordLSTM(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)

    def forward(self, x):
        embeds = self.embedding(x)
        lstm_out, _ = self.lstm(embeds)
        last_output = lstm_out[:, -1, :]
        prediction = self.fc(last_output)
        return prediction
```

Step 4: Train

```
model = NextWordLSTM(vocab_size=len(vocab), embed_size=32, hidden_size=64)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()

for epoch in range(100):
    for input_seq, target in sequences:
        input_tensor = torch.tensor([input_seq])
        target_tensor = torch.tensor([target])

        output = model(input_tensor)
```

```
    loss = criterion(output, target_tensor)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Step 5: Test

```
test_input = ["the", "cat", "sat"]
input_tensor = torch.tensor([[word_to_idx[w] for w in test_input]])
output = model(input_tensor)
predicted_idx = output.argmax(dim=1).item()
print(f"Predicted next word: {idx_to_word[predicted_idx]}")
# Should predict "on"!
```

What This Teaches You

- How sequences are processed one step at a time
- How LSTM maintains memory across steps
- How to train on sequential data
- The same concept applies to video frames in ExamGuard

Connection to ExamGuard

Next Word Predictor	ExamGuard
Input: 3 words	Input: 10 video frames
Each word = embedding vector	Each frame = CNN feature vector
LSTM reads word by word	LSTM reads frame by frame
Output: next word	Output: cheating or normal
Memory: remembers previous words	Memory: remembers previous behaviors

The skill is IDENTICAL — only the data type changes.

Key Takeaways

- 1. LSTM understands TIME — it does not just see one moment, it sees the whole story
- 2. Cheating is a sequence — no single frame proves it, but the pattern over time does
- 3. Hidden state = memory — LSTM remembers what it saw before
- 4. Three gates control memory — forget old stuff, remember new stuff, output relevant stuff
- 5. Start with text sequences — easier to understand before jumping to video

Autoencoders — Teaching AI What "Normal" Looks Like

What Is This?

Imagine you study 1000 photos of cats. You get SO good at drawing cats from memory that you can recreate any cat photo almost perfectly.

Now someone shows you a photo of a DOG and asks you to recreate it. You have never studied dogs. Your recreation looks TERRIBLE — it looks like a weird cat-dog hybrid.

That terrible recreation = the system telling you "this is NOT what I was trained on!"

An Autoencoder is a neural network that does exactly this:

- 1. Takes an input (like a video frame)
- 2. COMPRESSES it into a tiny representation
- 3. RECREATES the original from that tiny version
- 4. Compares the recreation to the original

If the recreation is good → the input was NORMAL (similar to training data).

If the recreation is bad → the input was ABNORMAL (different from training data).

How Does It Work?

The Architecture

```
INPUT (full image) → ENCODER → BOTTLENECK → DECODER → OUTPUT (recreated image)
   224x224          Shrink    Just 32      Expand     224x224
   = 150,528        down      numbers!     back up    = 150,528
   numbers                                 numbers
```

Think of It Like This

```
Original photo (high detail)
   |
   v
ENCODER: Summarize in 10 words
   |
   v
"Student sitting, writing, head down, pen moving, paper centered"
   |
   v
DECODER: Recreate photo from those 10 words
   |
   v
Recreated photo (close to original, but not perfect)
```

If the original was a NORMAL student writing, the 10-word summary captures it well, and recreation is close.

If the original was a student doing something WEIRD (standing on desk?), the 10-word summary cannot capture it, and recreation is terrible.

The Three Parts

1. Encoder (Compressor)

- Takes the full input
- Squeezes it down to a tiny representation
- Forces the network to learn what is IMPORTANT

2. Bottleneck (The Tiny Middle)

- Just a small number of values (like 32 or 64 numbers)
- This is the "compressed summary" of the input
- FORCES the network to learn only the ESSENTIAL patterns

3. Decoder (Reconstructor)

- Takes the tiny representation
- Tries to rebuild the original input
- The better the training, the better the reconstruction

WHY ExamGuard Needs This

The Problem: Unknown Cheating Methods

You can train a model to recognize specific cheating:

- Looking at neighbor (known method)
- Using phone (known method)
- Passing notes (known method)

But what about CREATIVE cheating you have NEVER seen?

- Tapping desk in Morse code to communicate
- Using a smartwatch disguised as regular watch
- Coughing patterns as signals
- Touching specific body parts as coded answers

You cannot train a classifier for methods you have not seen!

The Autoencoder Solution

Instead of learning "what cheating looks like," learn "what NORMAL looks like."

Training (before exam):
- Feed 10,000 clips of NORMAL student behavior
- Autoencoder learns to compress and recreate normal behavior perfectly

During exam:
- Feed live video frames
- Autoencoder tries to recreate each frame
- If recreation error is LOW → Normal behavior → Ignore
- If recreation error is HIGH → Something unusual → FLAG IT

ExamGuard Example

Normal behavior (trained on): Reconstruction error: 0.02 (LOW)
- Student writing → Normal. No alert.
- Student looking at own paper
- Student stretching occasionally

Unusual behavior (never trained on): Reconstruction error: 0.45 (HIGH)
- Student tapping desk rhythmically → FLAGGED! Unusual pattern.
- Student touching ear repeatedly → FLAGGED! Possible earpiece.
- Student making hand signals → FLAGGED! Unknown behavior.

The autoencoder does not need to know WHAT the cheating method is. It just knows "this is NOT normal."

Real-World Connection

This is exactly how credit card fraud detection works:

- 1. Bank learns your NORMAL spending pattern (coffee shop, grocery store, gas station)
- 2. Someone uses your card at a luxury store in another country
- 3. System says "I cannot reconstruct this as normal behavior" → FLAGGED

ExamGuard does the same thing but with VIDEO instead of transactions.

What You Need to Learn

Step 1: Basic Autoencoder Structure

```
import torch
import torch.nn as nn

class BasicAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()

        # ENCODER: Compress input
        self.encoder = nn.Sequential(
            nn.Linear(784, 256),    # 784 input features → 256
            nn.ReLU(),
            nn.Linear(256, 64),     # 256 → 64
            nn.ReLU(),
            nn.Linear(64, 32),      # 64 → 32 (bottleneck!)
            nn.ReLU()
        )

        # DECODER: Reconstruct from compressed
        self.decoder = nn.Sequential(
            nn.Linear(32, 64),      # 32 → 64
            nn.ReLU(),
            nn.Linear(64, 256),     # 64 → 256
            nn.ReLU(),
            nn.Linear(256, 784),    # 256 → 784 (same as input!)
            nn.Sigmoid()            # Output between 0 and 1
        )

    def forward(self, x):
        compressed = self.encoder(x)    # Compress
        reconstructed = self.decoder(compressed)  # Reconstruct
        return reconstructed
```

Step 2: Training the Autoencoder

```
model = BasicAutoencoder()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.MSELoss()  # Mean Squared Error — how different is output from input?

for epoch in range(50):
    for batch in normal_data_loader:  # ONLY normal data!
        # The input IS the target (we want to recreate the input)
        output = model(batch)
        loss = criterion(output, batch)  # Compare output to input

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch}: Reconstruction error = {loss.item():.4f}")
```

Step 3: Using It for Anomaly Detection

```
def detect_anomaly(model, new_data, threshold=0.1):
    """
    If reconstruction error > threshold → ANOMALY
    """
    model.eval()
    with torch.no_grad():
        reconstructed = model(new_data)
        error = torch.mean((new_data - reconstructed) ** 2, dim=1)

        for i, err in enumerate(error):
            if err > threshold:
                print(f"Sample {i}: ERROR={err:.4f} → ANOMALY DETECTED!")
            else:
                print(f"Sample {i}: ERROR={err:.4f} → Normal")
```

Step 4: Convolutional Autoencoder (For Images)

```
class ConvAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()

        # ENCODER: Image → compressed
        self.encoder = nn.Sequential(
            nn.Conv2d(3, 32, 3, stride=2, padding=1),   # 224→112
            nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1),  # 112→56
            nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), # 56→28
            nn.ReLU(),
            nn.Conv2d(128, 256, 3, stride=2, padding=1),# 28→14
            nn.ReLU()
        )

        # DECODER: Compressed → image
```

```
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(256, 128, 3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(128, 64, 3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(64, 32, 3, stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(32, 3, 3, stride=2, padding=1, output_padding=1),
            nn.Sigmoid()
        )

    def forward(self, x):
        compressed = self.encoder(x)
        reconstructed = self.decoder(compressed)
        return reconstructed
```

Step 5: Setting the Right Threshold

This is the HARDEST part. Too sensitive = too many false alarms. Too loose = misses anomalies.

```
# After training, measure error on NORMAL validation data
normal_errors = []
for batch in normal_validation_data:
    reconstructed = model(batch)
    error = torch.mean((batch - reconstructed) ** 2, dim=1)
    normal_errors.extend(error.tolist())

# Set threshold at 95th percentile of normal errors
# This means 5% of NORMAL data will be flagged (acceptable)
import numpy as np
threshold = np.percentile(normal_errors, 95)
print(f"Threshold set at: {threshold:.4f}")

# Anything with error ABOVE this threshold = anomaly
```

Mini Project: Anomaly Detection on Handwritten Digits

Goal

Train an autoencoder on the digit "1". Then test it with other digits — it should flag them as anomalies.

Step-by-Step

Step 1: Load Data

```
from torchvision import datasets, transforms

transform = transforms.Compose([
    transforms.ToTensor()
])

mnist = datasets.MNIST(root='./data', train=True, download=True, transform=transform)

# Filter only digit "1" for training (this is our "normal")
```

```
normal_data = [img for img, label in mnist if label == 1]
print(f"Training on {len(normal_data)} images of digit 1")
```

Step 2: Build Simple Autoencoder

```
class DigitAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28*28, 128),
            nn.ReLU(),
            nn.Linear(128, 32),  # Bottleneck: 32 numbers
            nn.ReLU()
        )
        self.decoder = nn.Sequential(
            nn.Linear(32, 128),
            nn.ReLU(),
            nn.Linear(128, 28*28),
            nn.Sigmoid(),
            nn.Unflatten(1, (1, 28, 28))
        )

    def forward(self, x):
        compressed = self.encoder(x)
        reconstructed = self.decoder(compressed)
        return reconstructed
```

Step 3: Train on Normal Data Only

```
# Train ONLY on digit "1"
model = DigitAutoencoder()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = nn.MSELoss()

for epoch in range(20):
    total_loss = 0
    for img in DataLoader(normal_data, batch_size=64, shuffle=True):
        output = model(img)
        loss = criterion(output, img)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch}: Loss = {total_loss/len(normal_data):.6f}")
```

Step 4: Test with Different Digits

```
# Test with digit 1 (normal) — should have LOW error
# Test with digit 7 (anomaly) — should have HIGH error
# Test with digit 3 (anomaly) — should have HIGH error

test_data = datasets.MNIST(root='./data', train=False, transform=transform)

for digit in [1, 3, 5, 7, 9]:
```

```
digit_samples = [img for img, label in test_data if label == digit][:100]
errors = []
for img in digit_samples:
    img = img.unsqueeze(0)
    reconstructed = model(img)
    error = torch.mean((img - reconstructed) ** 2).item()
    errors.append(error)

avg_error = sum(errors) / len(errors)
status = "NORMAL" if digit == 1 else "ANOMALY"
print(f"Digit {digit}: Avg Error = {avg_error:.4f} → Expected: {status}")
```

Expected Output:

```
Digit 1: Avg Error = 0.0023 → Expected: NORMAL    (low error — it knows this!)
Digit 3: Avg Error = 0.0187 → Expected: ANOMALY   (high error — never seen this)
Digit 5: Avg Error = 0.0215 → Expected: ANOMALY   (high error)
Digit 7: Avg Error = 0.0098 → Expected: ANOMALY   (medium — 7 looks a bit like 1)
Digit 9: Avg Error = 0.0145 → Expected: ANOMALY   (high error)
```

Connection to ExamGuard

Digit Autoencoder	**ExamGuard Autoencoder**
Trained on digit "1"	Trained on normal student behavior
Tests new digits	Tests live video frames
High error = not digit 1	High error = not normal behavior
Catches ANY non-1 digit	Catches ANY unusual behavior
No need to know what digit 3 looks like	No need to know what the cheating method is

Key Takeaways

- 1. Autoencoders learn to compress and recreate — the bottleneck forces them to learn patterns
- 2. Train on NORMAL only — the magic is that anomalies are detected automatically
- 3. Reconstruction error = anomaly score — high error means "I have never seen anything like this"
- 4. Catches UNKNOWN cheating methods — does not need to be trained on specific cheating
- 5. Threshold setting is critical — too low means too many false alarms, too high means missed catches

Anomaly Detection — Catching What You Have Never Seen Before

What Is This?

Anomaly detection means finding things that DO NOT FIT the normal pattern.

Think of a school classroom:

- 29 students wearing uniforms → NORMAL
- 1 student wearing a clown costume → ANOMALY

You did not need to be trained on "clown costumes." You just know it does not fit.

Anomaly detection in AI works the same way. Train on normal. Anything different = flagged.

WHY ExamGuard Needs This

The Limitation of Classification

A classifier says: "Is this cheating method A, B, or C?"

But what about cheating method D, E, F that you NEVER trained on?

Trained to detect:	Cannot detect:
- Looking at neighbor	- Morse code tapping
- Using phone	- Smartwatch cheating
- Passing notes	- Coded cough patterns
- Hidden earpiece	- Invisible ink
- Writing on hand	- Shoe camera

Cheaters are CREATIVE. New methods appear every exam season.

The Anomaly Detection Solution

Instead of asking "Is this cheating method X?" ask "Is this NORMAL?"

Normal student behavior:
- Writing on paper
- Looking at own paper
- Occasional look up (thinking)
- Stretching
- Drinking water
- Flipping pages

ANYTHING significantly different from this list → FLAG IT

Now it catches EVERYTHING unusual, including methods you never imagined.

Three Approaches to Anomaly Detection

Approach 1: Autoencoder-Based (Best for ExamGuard)

You already learned this in the previous file. Quick recap:

Train autoencoder on normal behavior
↓
Feed new video frame
↓
Measure reconstruction error

↓
High error = anomaly

Best for: Image and video data (ExamGuard's main use case)

Approach 2: Isolation Forest

The idea: Normal data points are SURROUNDED by other normal points. Anomalies are ISOLATED — far away from everything else.

Imagine a scatter plot:

```
Normal points:      Anomaly:
* * * *                * (all alone!)
* * * * *
* * * * *
* * * *
```

The lonely point is easy to ISOLATE — it takes fewer splits to separate it.

How it works:

- 1. Randomly pick a feature (like "head angle")
- 2. Randomly pick a split value
- 3. Repeat until each point is isolated
- 4. Points that get isolated QUICKLY = anomalies (they are far from others)
- 5. Points that take MANY splits to isolate = normal (surrounded by similar points)

```
from sklearn.ensemble import IsolationForest

# Features: [head_angle, body_lean, hand_position, gaze_direction]
normal_data = [...]  # 10,000 samples of normal behavior features

# Train
model = IsolationForest(contamination=0.05)  # Expect 5% anomalies
model.fit(normal_data)

# Test
new_sample = [45, 12, 3, 7]  # Features from a new frame
result = model.predict([new_sample])
# result = 1 → Normal
# result = -1 → ANOMALY
```

Best for: Tabular/numerical data (feature vectors extracted from video)

Approach 3: One-Class SVM

The idea: Draw a BOUNDARY around all normal data. Anything outside the boundary = anomaly.

Think of it like drawing a circle around normal data:

```
   /----------\
  / * * * *    \
 | * * * * *    |   X ← Anomaly (outside circle)
 | * * * * *    |
```

```
    \* * * *   /
     \----------/
```

```
from sklearn.svm import OneClassSVM

# Train on normal data only
model = OneClassSVM(kernel='rbf', gamma='auto', nu=0.05)
model.fit(normal_data)

# Test
result = model.predict([new_sample])
# result = 1 → Inside boundary → Normal
# result = -1 → Outside boundary → ANOMALY
```

Best for: When you have clean normal data and clear boundaries

ExamGuard: Combining All Three

In a real system, you do not pick just one. You COMBINE them:

```
Video frame
  ↓
Extract features (CNN)
  ↓
┌──────────────────────────────────────────┐
│ Autoencoder: Reconstruction error = 0.3 │ → Suspicious
│ Isolation Forest: Score = -0.8          │ → Anomaly
│ One-Class SVM: Prediction = -1          │ → Anomaly
└──────────────────────────────────────────┘
  ↓
2 out of 3 say anomaly → FLAG IT
```

This "ensemble" approach reduces false alarms because all methods must agree.

The Biggest Challenge: Setting the Threshold

Too Sensitive (Threshold Too Low)

Threshold: 0.05

```
Normal student scratches head    → Error: 0.06 → ALERT!  (false alarm)
Normal student drops pen         → Error: 0.07 → ALERT!  (false alarm)
Normal student yawns             → Error: 0.08 → ALERT!  (false alarm)
Actual cheater passing notes     → Error: 0.15 → ALERT!  (correct)

Result: 100 alerts per hour, 97 are false alarms.
Invigilator ignores ALL alerts because too many false ones.
The 3 real ones get missed!
```

Too Loose (Threshold Too High)

Threshold: 0.50

```
Normal student scratches head    → Error: 0.06 → OK     (correct)
Student tapping Morse code       → Error: 0.25 → OK     (MISSED!)
Student using hidden earpiece    → Error: 0.35 → OK     (MISSED!)
```

Student openly copying → Error: 0.65 → ALERT! (correct)

Result: Only catches obvious cheating. Subtle methods slip through.

The Sweet Spot

Threshold: 0.15

Normal student scratches head → Error: 0.06 → OK (correct)
Normal student drops pen → Error: 0.07 → OK (correct)
Student tapping Morse code → Error: 0.25 → ALERT! (correct)
Student using hidden earpiece → Error: 0.35 → ALERT! (correct)

Result: Catches unusual behavior, ignores normal quirks.
Maybe 5-10 alerts per hour, most are worth investigating.

How to Find the Right Threshold

```
import numpy as np

# Step 1: Get reconstruction errors for ALL normal validation data
normal_errors = []
for sample in normal_validation_set:
    error = calculate_reconstruction_error(model, sample)
    normal_errors.append(error)

# Step 2: Look at the distribution
print(f"Mean error: {np.mean(normal_errors):.4f}")
print(f"Std error: {np.std(normal_errors):.4f}")
print(f"95th percentile: {np.percentile(normal_errors, 95):.4f}")
print(f"99th percentile: {np.percentile(normal_errors, 99):.4f}")

# Step 3: Set threshold at 95th or 99th percentile
# 95th → 5% of normal data flagged (more sensitive, more false alarms)
# 99th → 1% of normal data flagged (less sensitive, fewer false alarms)
threshold = np.percentile(normal_errors, 97)  # 3% false alarm rate
```

Real ExamGuard Anomaly Examples

True Anomalies (Should Be Caught)

Behavior	Why It Is Anomalous
Rhythmic desk tapping	No normal student taps for 30+ seconds
Repeated ear touching	Normal = occasional scratch. This = every 10 seconds
Hand under desk for extended time	Normal = adjusting. This = 2+ minutes
Synchronized movements with neighbor	Two students moving in sync = coordination
Looking at same spot on wall	Could be hidden notes on wall
Frequent bathroom requests	3+ times in 1 hour exam

False Alarms (Should NOT Be Caught)

Behavior	Why It Looks Anomalous But Is Not
Student with disability	Different posture is NORMAL for them

```
Student who fidgets              Some people naturally fidget a lot
Student having anxiety attack    Unusual behavior but not cheating
Left-handed student              Different arm position than most
Student wearing cast             Different body movement
Very tall student                Looks different in frame
```

This is why human-in-the-loop is ESSENTIAL. AI flags, human decides.

What You Need to Learn

Concept 1: What Makes a Good "Normal" Dataset

- Must be LARGE (10,000+ samples minimum)
- Must be DIVERSE (different students, different halls, different lighting)
- Must be CLEAN (no cheating behavior mixed in)
- Must represent ALL normal behaviors (writing, thinking, stretching, etc.)

Concept 2: Feature Engineering for Anomaly Detection

```
# What features to extract from each frame?
features = {
   'head_angle_x': 15.2,       # Head tilt left/right
   'head_angle_y': -5.1,       # Head tilt up/down
   'body_lean': 3.4,           # How much body is leaning
   'hand_position_x': 120,     # Where is the dominant hand
   'hand_position_y': 85,      # Vertical hand position
   'gaze_direction': 'paper',  # Where student is looking
   'movement_speed': 2.1,      # How fast student is moving
   'mouth_open': False,        # Is mouth open (talking?)
   'neighbor_distance': 45,    # Distance to nearest neighbor
}
```

Concept 3: Evaluation Metrics

```
True Positive  (TP): AI says anomaly, it IS anomaly     → GOOD
True Negative  (TN): AI says normal, it IS normal       → GOOD
False Positive (FP): AI says anomaly, it is NORMAL      → BAD (false alarm)
False Negative (FN): AI says normal, it IS anomaly      → VERY BAD (missed cheating)

Precision = TP / (TP + FP)  → "When AI says anomaly, how often is it right?"
Recall    = TP / (TP + FN)  → "Of all real anomalies, how many did AI catch?"

For ExamGuard: Recall is MORE important than precision.
Missing a cheater (FN) is WORSE than a false alarm (FP).
```

Mini Project: Credit Card Fraud Detection

This is the CLASSIC anomaly detection problem. Perfect practice for ExamGuard.

Goal

Detect fraudulent credit card transactions using anomaly detection.

Step-by-Step

Step 1: Get the Data

```
# Kaggle dataset: "Credit Card Fraud Detection"
# Download from: https://www.kaggle.com/mlg-ulb/creditcardfraud
import pandas as pd
from sklearn.model_selection import train_test_split

data = pd.read_csv('creditcard.csv')
print(f"Total transactions: {len(data)}")
print(f"Fraud cases: {data['Class'].sum()}")
print(f"Fraud percentage: {data['Class'].mean()*100:.2f}%")
# Usually around 0.17% — very imbalanced, just like ExamGuard!
```

Step 2: Prepare Data

```
# Separate normal and fraud
normal = data[data['Class'] == 0]
fraud = data[data['Class'] == 1]

# Use only normal data for training (anomaly detection style)
X_train = normal.drop('Class', axis=1).values[:200000]  # Train on normal
X_val_normal = normal.drop('Class', axis=1).values[200000:]  # Validate normal
X_val_fraud = fraud.drop('Class', axis=1).values  # Validate fraud
```

Step 3: Try Isolation Forest

```
from sklearn.ensemble import IsolationForest
from sklearn.metrics import classification_report

model = IsolationForest(contamination=0.01, random_state=42)
model.fit(X_train)

# Test on normal data
normal_preds = model.predict(X_val_normal)
print(f"Normal detected as normal: {(normal_preds == 1).sum()}")
print(f"Normal detected as fraud (false alarms): {(normal_preds == -1).sum()}")

# Test on fraud data
fraud_preds = model.predict(X_val_fraud)
print(f"Fraud detected as fraud: {(fraud_preds == -1).sum()}")
print(f"Fraud detected as normal (MISSED): {(fraud_preds == 1).sum()}")
```

Step 4: Try Autoencoder

```
import torch
import torch.nn as nn

class FraudAutoencoder(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU()
```

```
        )
        self.decoder = nn.Sequential(
            nn.Linear(16, 32),
            nn.ReLU(),
            nn.Linear(32, 64),
            nn.ReLU(),
            nn.Linear(64, input_dim),
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

# Train on normal transactions only
# Test: fraud transactions should have HIGH reconstruction error
```

Step 5: Compare Methods

```
# Which method catches more fraud with fewer false alarms?
# Try: Isolation Forest, One-Class SVM, Autoencoder
# Measure precision and recall for each
# The best one = your go-to for ExamGuard
```

What This Teaches You

- Working with imbalanced data (rare events, just like cheating)
- Setting and tuning thresholds
- Measuring precision vs recall trade-off
- Comparing different anomaly detection methods

Connection to ExamGuard

Credit Card Fraud	**ExamGuard**
Normal transactions = purchases	Normal behavior = writing, reading
Fraud = unusual transactions	Cheating = unusual behavior
Train on normal transactions	Train on normal exam footage
High reconstruction error = fraud	High reconstruction error = suspicious
0.17% fraud rate	Maybe 1-5% cheating rate
Cannot predict new fraud types	Cannot predict new cheating methods
Anomaly detection catches new types	Anomaly detection catches new methods

Key Takeaways

- 1. Anomaly detection finds the UNKNOWN — catches cheating methods never seen before
- 2. Three main approaches — Autoencoder (images), Isolation Forest (features), One-Class SVM (boundaries)
- 3. Threshold is everything — too sensitive means too many false alarms, too loose means missed catches
- 4. Combine multiple methods — ensemble approach reduces errors
- 5. Human-in-the-loop is essential — AI flags, human investigates and decides
- 6. Start with credit card fraud — classic problem, same concepts apply to ExamGuard

Reinforcement Learning — Training the Decision-Making Brain

What Is This?

Every other AI model you have learned so far answers ONE question:

- CNN: "What is in this image?"
- LSTM: "What is happening over time?"
- Autoencoder: "Is this normal or abnormal?"

But none of them answer the REAL question:

"What should I DO about it?"

Should the system:

- Send an alert NOW?
- Wait and watch more?
- Mark it as low priority?
- Sound a high-priority alarm?
- Ignore it completely?

Reinforcement Learning (RL) trains an agent to make DECISIONS by trying things and learning from the results.

How Does It Work?

The Concept: Learning Like a Child

A child touches a hot stove:

Action: Touch stove
Result: PAIN (-100 reward)
Lesson: Do NOT touch stove again

A child eats candy:

Action: Eat candy
Result: DELICIOUS (+50 reward)
Lesson: Eat candy when available

The child learns WHICH ACTIONS lead to GOOD results and which lead to BAD results.

RL works the SAME way:

Agent (the AI) is in a SITUATION (state)
Agent takes an ACTION
Environment gives a REWARD (positive or negative)
Agent learns which actions give the best rewards

The Four Key Parts

1. AGENT = The decision maker (ExamGuard's alert system)
2. STATE = Current situation (what the cameras are showing)
3. ACTION = What the agent decides to do (alert, ignore, watch more)
4. REWARD = Feedback on how good the action was (+100 or -50)

WHY ExamGuard Needs This

The Problem: Too Many Decisions

Every second, the system sees dozens of behaviors across all cameras. For each one:

Student glanced at neighbor for 0.5 seconds
→ Should I alert? Most invigilators would say NO — it is a quick glance.

Student stared at neighbor for 5 seconds
→ Should I alert? Maybe — it is getting suspicious.

Student stared at neighbor for 5 seconds AND leaned toward them
→ Should I alert? YES — this is very likely cheating.

Student stared at neighbor AND neighbor covered their paper
→ Should I alert? DEFINITELY — both students involved.

A simple threshold (like "alert if confidence > 80%") is too crude. The CONTEXT matters.

The RL Solution

Train an agent that learns the BEST action for every situation:

State: [gaze_at_neighbor=0.5sec, lean=none, hand=writing]
Agent's learned action: IGNORE (just a quick glance)
Reward: +10 (correct — it was indeed innocent)

State: [gaze_at_neighbor=5sec, lean=toward_neighbor, hand=not_writing]
Agent's learned action: HIGH PRIORITY ALERT
Reward: +100 (correct — invigilator confirmed cheating)

State: [gaze_at_clock=3sec, lean=back, hand=stretching]
Agent's learned action: IGNORE
Reward: +10 (correct — student was just checking time)

Over thousands of training episodes, the agent learns the PERFECT policy.

ExamGuard Reward System

Actions the Agent Can Take

Action 0: IGNORE → Do nothing, keep monitoring
Action 1: WATCH_CLOSER → Increase monitoring on this student
Action 2: LOW_ALERT → Flag to invigilator as "worth a look"
Action 3: MEDIUM_ALERT → Flag as "probably suspicious"
Action 4: HIGH_ALERT → Flag as "very likely cheating — check immediately"

Reward Values

Correct alert (AI alerts, human confirms cheating):
HIGH_ALERT on real cheating → +100
MEDIUM_ALERT on real cheating → +70
LOW_ALERT on real cheating → +30

Correct silence (AI ignores, nothing was wrong):

IGNORE on normal behavior → +10

False alarm (AI alerts, but nothing was wrong):
HIGH_ALERT on normal behavior → -50 (wastes invigilator time)
MEDIUM_ALERT on normal → -30
LOW_ALERT on normal → -10

Missed cheating (AI ignores, but it WAS cheating):
IGNORE on real cheating → -200 (WORST outcome!)

Why These Numbers?

- Missing cheating (-200) is MUCH worse than a false alarm (-50)
- So the agent learns to be CAUTIOUS — better to flag and be wrong than miss real cheating
- But false alarms still have a penalty, so it does not alert on everything
- WATCH_CLOSER has no penalty — it is a safe "middle ground"

What the Agent Learns

After training, the agent develops nuanced behavior:

Scenario 1: Quick glance at neighbor (0.5 seconds)
State: [gaze_duration=0.5, lean=0, hand_movement=writing]
Action: IGNORE
Why: Too brief, student went right back to writing

Scenario 2: Extended stare at neighbor (5 seconds)
State: [gaze_duration=5.0, lean=0, hand_movement=stopped]
Action: WATCH_CLOSER
Why: Suspicious but not enough evidence yet

Scenario 3: Stare + lean toward neighbor
State: [gaze_duration=5.0, lean=15_degrees, hand_movement=stopped]
Action: MEDIUM_ALERT
Why: Multiple indicators suggest cheating

Scenario 4: Stare + lean + neighbor covers paper
State: [gaze_duration=5.0, lean=15, neighbor_covering=True]
Action: HIGH_ALERT
Why: Very strong evidence — both students involved

Scenario 5: Student looks around room frequently but briefly
State: [gaze_changes=high, duration_each=0.3, pattern=random]
Action: IGNORE
Why: Probably just anxious/nervous, not targeting anyone

How to Implement RL

Step 1: Define the Environment

```
import gymnasium as gym
import numpy as np
```

```
class ExamGuardEnv(gym.Env):
  """
  Custom environment for ExamGuard alert decisions.
  """
  def __init__(self):
    super().__init__()

    # State space: what the agent sees
    # [gaze_duration, lean_angle, hand_activity, mouth_movement,
    #  neighbor_distance, time_since_last_alert, confidence_score]
    self.observation_space = gym.spaces.Box(
      low=np.array([0, -30, 0, 0, 0, 0, 0]),
      high=np.array([30, 30, 5, 1, 100, 300, 1]),
      dtype=np.float32
    )

    # Action space: what the agent can do
    # 0=ignore, 1=watch, 2=low alert, 3=medium alert, 4=high alert
    self.action_space = gym.spaces.Discrete(5)

  def reset(self, seed=None):
    """Start a new monitoring episode."""
    # Generate a random scenario
    self.state = self._generate_scenario()
    self.is_cheating = self._determine_ground_truth()
    return self.state, {}

  def step(self, action):
    """Agent takes an action, environment gives reward."""
    reward = self._calculate_reward(action, self.is_cheating)

    # Move to next time step
    self.state = self._generate_scenario()
    self.is_cheating = self._determine_ground_truth()

    done = False  # Episode continues
    return self.state, reward, done, False, {}

  def _calculate_reward(self, action, is_cheating):
    """Reward based on action and ground truth."""
    if is_cheating:
      # Cheating IS happening
      rewards = {
        0: -200,   # Ignored cheating — TERRIBLE
        1: -50,    # Just watching — not enough
        2: +30,    # Low alert — caught it, but low urgency
        3: +70,    # Medium alert — good
        4: +100,   # High alert — perfect response
      }
    else:
      # Nothing is happening (normal behavior)
```

```
        rewards = {
            0: +10,   # Correctly ignored — good
            1: +5,    # Watching closer — slight waste but OK
            2: -10,   # Low alert — minor false alarm
            3: -30,   # Medium alert — wasting time
            4: -50,   # High alert — major false alarm
        }
    return rewards[action]
```

Step 2: Train Using Stable Baselines3

```
from stable_baselines3 import PPO

# Create environment
env = ExamGuardEnv()

# Create RL agent (PPO algorithm — good general-purpose RL)
model = PPO(
    "MlpPolicy",     # Use a simple neural network
    env,
    verbose=1,
    learning_rate=0.0003,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99       # How much agent values future rewards
)

# Train for 100,000 steps
model.learn(total_timesteps=100_000)

# Save the trained agent
model.save("examguard_alert_agent")
```

Step 3: Use the Trained Agent

```
# Load trained agent
model = PPO.load("examguard_alert_agent")

# In real-time: get state from cameras, ask agent what to do
state = get_current_state_from_cameras()  # [gaze, lean, hand, ...]

action, _ = model.predict(state)

action_names = {0: "IGNORE", 1: "WATCH", 2: "LOW ALERT",
                3: "MEDIUM ALERT", 4: "HIGH ALERT"}
print(f"Agent decision: {action_names[action]}")
```

Step 4: Improve with Real Feedback

```
# After pilot testing, use real invigilator feedback as rewards
# "Was this alert correct?" → Yes (+100) or No (-50)
# Retrain the agent with this real-world feedback
# Each cycle makes the agent smarter
```

Libraries You Need

Stable Baselines3

pip install stable-baselines3

- Pre-built RL algorithms (PPO, DQN, A2C, SAC)
- Easy to use, well documented
- Works with custom environments

OpenAI Gymnasium

pip install gymnasium

- Framework for creating RL environments
- Standard interface that all RL libraries use
- Comes with practice environments (CartPole, etc.)

Mini Project: CartPole Balancing

Before building ExamGuard's RL system, learn RL basics with the classic CartPole problem.

What Is CartPole?

A pole is balanced on a cart. You can push the cart LEFT or RIGHT. Goal: keep the pole from falling.

```
    |
    |  ← Pole (keep this upright!)
 ____|____
|  CART   |
|_________|
←  LEFT   RIGHT  →
```

Step-by-Step

Step 1: Install Libraries

```
pip install stable-baselines3 gymnasium
```

Step 2: See the Environment

```
import gymnasium as gym

env = gym.make("CartPole-v1", render_mode="human")

state, info = env.reset()
print(f"State: {state}")
# State = [cart_position, cart_velocity, pole_angle, pole_velocity]

for _ in range(100):
   action = env.action_space.sample()  # Random action (0=left, 1=right)
   state, reward, done, truncated, info = env.step(action)
   if done:
      state, info = env.reset()

env.close()
# You will see the cart moving randomly and the pole falling quickly
```

Step 3: Train an RL Agent

```
from stable_baselines3 import PPO

env = gym.make("CartPole-v1")

# Create agent
model = PPO("MlpPolicy", env, verbose=1)

# Train (this takes about 1 minute)
model.learn(total_timesteps=50_000)

print("Training complete!")
```

Step 4: Watch the Trained Agent

```
env = gym.make("CartPole-v1", render_mode="human")

state, info = env.reset()
total_reward = 0

for _ in range(500):
    action, _ = model.predict(state)  # Agent decides (not random!)
    state, reward, done, truncated, info = env.step(action)
    total_reward += reward
    if done:
        print(f"Episode reward: {total_reward}")
        state, info = env.reset()
        total_reward = 0

env.close()
# The pole should stay balanced much longer now!
```

Step 5: Compare Before and After

```
# Test random agent (no training)
random_rewards = []
for _ in range(100):
    state, _ = env.reset()
    episode_reward = 0
    done = False
    while not done:
        action = env.action_space.sample()
        state, reward, done, _, _ = env.step(action)
        episode_reward += reward
    random_rewards.append(episode_reward)

print(f"Random agent average: {sum(random_rewards)/len(random_rewards):.1f}")
# Usually around 20-30

# Test trained agent
trained_rewards = []
for _ in range(100):
    state, _ = env.reset()
```

```
    episode_reward = 0
    done = False
    while not done:
        action, _ = model.predict(state)
        state, reward, done, _, _ = env.step(action)
        episode_reward += reward
    trained_rewards.append(episode_reward)

print(f"Trained agent average: {sum(trained_rewards)/len(trained_rewards):.1f}")
# Usually 400-500 (maximum is 500)
```

What This Teaches You

CartPole	ExamGuard
State: cart position, pole angle	State: gaze, lean, hand movement
Actions: push left or right	Actions: ignore, watch, alert
Reward: +1 for each step balanced	Reward: +100 for correct alert
Penalty: episode ends if pole falls	Penalty: -200 for missed cheating
Goal: keep pole balanced longest	Goal: make best alert decisions

Key Concepts to Master

1. Policy

The agent's STRATEGY — "given this state, what action should I take?"

```
Policy: If gaze_duration > 3 AND lean > 10 → MEDIUM_ALERT
        If gaze_duration < 1 → IGNORE
```

2. Value Function

"How good is this state?" — estimated total future reward from here.

```
State where student is writing normally → High value (safe, no penalty coming)
State where student staring at neighbor → Low value (missed cheating penalty coming)
```

3. Exploration vs Exploitation

- Exploration: Try random actions to discover new strategies
- Exploitation: Use the best known strategy
- Balance: Start with lots of exploration, gradually shift to exploitation

4. Discount Factor (Gamma)

- How much the agent values FUTURE rewards vs IMMEDIATE rewards
- Gamma = 0.99: Future matters almost as much as now
- Gamma = 0.5: Agent is short-sighted
- ExamGuard: Use high gamma (0.99) because missed cheating penalty comes LATER

Key Takeaways

- 1. RL learns DECISIONS, not classifications — it decides WHAT TO DO, not just what it sees
- 2. Reward design is critical — the rewards define what the agent optimizes for
- 3. Missing cheating must be penalized MORE than false alarms — this shapes cautious behavior
- 4. The agent learns nuance — quick glance vs long stare vs stare+lean get different responses

- 5. Start with CartPole — learn the RL framework before building the ExamGuard environment
- 6. Stable Baselines3 + Gymnasium — the standard tools for RL in Python

Real-Time Inference — Making AI Fast Enough for Live Video

What Is This?

You have trained amazing models. They can detect objects, track gaze, spot anomalies, and make decisions. In a Jupyter notebook, they work great.

But here is the problem:

Your model takes 500ms (0.5 seconds) to process ONE frame.
Camera sends 30 frames per second.
30 frames x 500ms = 15,000ms = 15 seconds needed per second of video.

You are 15x TOO SLOW. You can never catch up.

Real-time inference means making your models fast enough to process video AS IT HAPPENS — at 30 frames per second.

The Speed Requirement

The Math

Camera sends: 30 frames per second (fps)
Time available per frame: 1000ms / 30 = 33.3 milliseconds

Your ENTIRE pipeline must complete in under 33ms per frame:
- Read frame from camera: ~2ms
- YOLO object detection: ~8ms
- CNN behavior classification: ~5ms
- LSTM sequence analysis: ~3ms
- Autoencoder anomaly check: ~4ms
- RL decision making: ~1ms
- Send alert if needed: ~1ms

Total: ~24ms ← Under 33ms! We are good!

If ANY step is too slow, the whole pipeline falls behind.

What Happens When Too Slow?

Real time: Frame 1, 2, 3, 4, 5, 6, 7, 8, 9, 10...

Fast system:
Processing: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 (keeps up)
Alert: "Cheating at frame 5!" (detected in real-time)

Slow system:
Processing: 1, ...2, ...3, ...4 (still processing 4 when frame 10 happens)
Alert: "Cheating at frame 5!" (detected 6 frames LATE)
By then, cheating already happened. Evidence may be gone.

WHY ExamGuard Needs This

The Scale Problem

One camera is manageable. But ExamGuard needs to handle MANY:

Small exam: 4 cameras × 30 fps = 120 frames/second
Medium exam: 20 cameras × 30 fps = 600 frames/second
Large exam: 200 cameras × 30 fps = 6,000 frames/second

6,000 frames per second. Each needs full AI pipeline.
At 33ms per frame, you need: 6000 × 33ms = 198,000ms of compute per second.
That is 198 seconds of work needed every 1 second.
You need ~200 parallel processing units (GPU cores handle this).

Real-Time Matters

An invigilator watching live needs INSTANT alerts:

- Student pulls out phone → Alert should appear within 1-2 seconds
- Student starts copying → Flag immediately
- If alert comes 30 seconds late, the student already put the phone away

Optimization Techniques

Technique 1: Model Quantization

What: Convert model from 32-bit numbers to 8-bit numbers.

Result: 4x smaller, 2-4x faster, tiny accuracy loss.

Regular model (FP32):
  Weight: 0.123456789012345 (32 bits per number)
  Size: 200 MB
  Speed: 15ms per frame

Quantized model (INT8):
  Weight: 0.12 (8 bits per number)
  Size: 50 MB
  Speed: 5ms per frame

Accuracy drop: maybe 0.5% — almost nothing!

```
import torch

# Load your trained model
model = torch.load("examguard_model.pth")

# Quantize to INT8
quantized_model = torch.quantization.quantize_dynamic(
    model,
    {torch.nn.Linear, torch.nn.Conv2d},
    dtype=torch.qint8
)

# Compare sizes
import os
torch.save(model, "original.pth")
torch.save(quantized_model, "quantized.pth")
```

```
print(f"Original: {os.path.getsize('original.pth') / 1e6:.1f} MB")
print(f"Quantized: {os.path.getsize('quantized.pth') / 1e6:.1f} MB")
```

Technique 2: ONNX Export

What: Convert PyTorch model to a universal format that runs faster.

Result: 1.5-3x speedup, works on any platform.

```
import torch

model = torch.load("examguard_model.pth")
model.eval()

# Create a dummy input (same shape as real input)
dummy_input = torch.randn(1, 3, 224, 224)

# Export to ONNX
torch.onnx.export(
    model,
    dummy_input,
    "examguard_model.onnx",
    input_names=["image"],
    output_names=["prediction"],
    dynamic_axes={"image": {0: "batch_size"}}
)

# Now run with ONNX Runtime (faster than PyTorch)
import onnxruntime as ort

session = ort.InferenceSession("examguard_model.onnx")
result = session.run(None, {"image": dummy_input.numpy()})
```

Technique 3: TensorRT (NVIDIA GPU Optimization)

What: NVIDIA's tool that optimizes models specifically for their GPUs.

Result: 2-5x speedup on NVIDIA GPUs.

```
# Convert ONNX to TensorRT
# This is usually done via command line
# trtexec --onnx=examguard_model.onnx --saveEngine=examguard_model.trt

# Or in Python
import tensorrt as trt

# TensorRT fuses layers, optimizes memory, uses GPU-specific tricks
# Result: The fastest possible inference on NVIDIA hardware
```

Technique 4: Batch Processing

What: Process multiple frames at once instead of one at a time.

Result: GPU utilization goes from 30% to 90%.

```
# SLOW: One frame at a time
for frame in frames:
```

```
    result = model(frame)  # GPU does tiny job, wastes capacity

# FAST: Batch of 8 frames at once
batch = torch.stack(frames[:8])
results = model(batch)  # GPU does big job, fully utilized
```

Single frame: 8ms per frame (GPU barely working)
Batch of 8:   20ms for all 8 = 2.5ms per frame (GPU fully utilized)
Speedup: 3.2x!

Technique 5: Smart Frame Skipping

What: You do NOT need to process EVERY frame. Skip some.

```
# 30 fps camera, but we process every 3rd frame
# Result: 10 fps processing (still catches cheating, 3x less work)

frame_count = 0
while True:
    frame = camera.read()
    frame_count += 1

    if frame_count % 3 != 0:
        continue  # Skip this frame

    # Process only every 3rd frame
    result = model(frame)
```

Why this works:

- Cheating is not instantaneous — it takes several seconds
- 10 fps is still 10 chances per second to catch it
- Reduces computation by 67%

Technique 6: Pipeline Parallelism

What: While one model processes frame N, another model processes frame N-1.

```
Without pipeline:
  Frame 1: [YOLO 8ms][CNN 5ms][LSTM 3ms] = 16ms total, then start frame 2

With pipeline:
  Frame 1: [YOLO 8ms][CNN 5ms][LSTM 3ms]
  Frame 2:         [YOLO 8ms][CNN 5ms][LSTM 3ms]
  Frame 3:                 [YOLO 8ms][CNN 5ms][LSTM 3ms]

  Each frame still takes 16ms, but a new result comes every 8ms!
  Effective throughput: 125 fps instead of 62 fps
```

```
import threading
from queue import Queue

# Create queues between pipeline stages
yolo_queue = Queue(maxsize=10)
cnn_queue = Queue(maxsize=10)
```

```
def yolo_worker():
    while True:
        frame = camera_queue.get()
        detections = yolo_model(frame)
        yolo_queue.put((frame, detections))

def cnn_worker():
    while True:
        frame, detections = yolo_queue.get()
        behaviors = cnn_model(frame, detections)
        cnn_queue.put((frame, detections, behaviors))

def lstm_worker():
    while True:
        frame, detections, behaviors = cnn_queue.get()
        sequence_result = lstm_model(behaviors)
        process_alert(sequence_result)

# Run all workers in parallel
threading.Thread(target=yolo_worker, daemon=True).start()
threading.Thread(target=cnn_worker, daemon=True).start()
threading.Thread(target=lstm_worker, daemon=True).start()
```

ExamGuard Full Pipeline Timing

Target: Process One Camera at 30 FPS

Step	Time	Technique
Read frame	2ms	OpenCV optimized
Resize/preprocess	1ms	GPU preprocessing
YOLO detection	8ms	YOLOv8-small + TensorRT
Crop detected persons	1ms	GPU crop
CNN behavior classify	5ms	ResNet18 quantized
LSTM sequence check	3ms	Every 10th frame only
Autoencoder anomaly	4ms	Runs on separate GPU stream
RL decision	1ms	Tiny model, CPU is fine
Send alert	1ms	Async, non-blocking
TOTAL	26ms	Under 33ms budget!

Scaling to Multiple Cameras

1 GPU (NVIDIA RTX 3080):
- Can handle ~4 cameras at full pipeline
- Or ~12 cameras with frame skipping (every 3rd frame)

1 GPU (NVIDIA A100 — data center):
- Can handle ~20 cameras at full pipeline
- Or ~60 cameras with frame skipping

Large deployment (200 cameras):

- Need 8-10 GPUs
- Or use edge computing (process at camera, send only suspicious frames)

What You Need to Learn

Benchmarking Basics

```
import time

def benchmark_model(model, input_data, num_runs=100):
    """Measure how fast a model runs."""
    # Warm up (first run is always slow)
    for _ in range(10):
        model(input_data)

    # Measure
    start = time.time()
    for _ in range(num_runs):
        model(input_data)
    end = time.time()

    avg_time = (end - start) / num_runs * 1000  # Convert to ms
    fps = 1000 / avg_time

    print(f"Average inference time: {avg_time:.1f}ms")
    print(f"Frames per second: {fps:.1f}")
    print(f"Can handle {fps/30:.0f} cameras at 30fps")

    return avg_time
```

CPU vs GPU Comparison

```
import torch
import torchvision.models as models

model = models.resnet18(pretrained=True)
model.eval()
input_tensor = torch.randn(1, 3, 224, 224)

# CPU benchmark
model_cpu = model.cpu()
input_cpu = input_tensor.cpu()
cpu_time = benchmark_model(model_cpu, input_cpu)

# GPU benchmark
if torch.cuda.is_available():
    model_gpu = model.cuda()
    input_gpu = input_tensor.cuda()
    gpu_time = benchmark_model(model_gpu, input_gpu)

    print(f"\nGPU is {cpu_time/gpu_time:.1f}x faster than CPU")
```

Mini Project: Benchmark Your YOLO Model

Goal

Measure your YOLO model's speed with and without GPU. See if it meets the 33ms target.

Steps

Step 1: Load YOLO

```
from ultralytics import YOLO
import cv2
import time

model = YOLO("yolov8n.pt")  # nano version (fastest)
```

Step 2: Benchmark on CPU

```
# Use a test image
image = cv2.imread("test_image.jpg")

# Warm up
for _ in range(5):
    model(image, device='cpu', verbose=False)

# Measure
times = []
for _ in range(50):
    start = time.time()
    results = model(image, device='cpu', verbose=False)
    end = time.time()
    times.append((end - start) * 1000)

print(f"CPU - Average: {sum(times)/len(times):.1f}ms")
print(f"CPU - FPS: {1000/(sum(times)/len(times)):.1f}")
```

Step 3: Benchmark on GPU

```
# Warm up
for _ in range(5):
    model(image, device=0, verbose=False)  # device=0 means first GPU

# Measure
times = []
for _ in range(50):
    start = time.time()
    results = model(image, device=0, verbose=False)
    end = time.time()
    times.append((end - start) * 1000)

print(f"GPU - Average: {sum(times)/len(times):.1f}ms")
print(f"GPU - FPS: {1000/(sum(times)/len(times)):.1f}")
```

Step 4: Compare Different YOLO Sizes

```
models_to_test = {
    "YOLOv8-nano":   YOLO("yolov8n.pt"),   # Fastest, least accurate
```

```
    "YOLOv8-small":  YOLO("yolov8s.pt"),   # Good balance
    "YOLOv8-medium": YOLO("yolov8m.pt"),   # More accurate, slower
    "YOLOv8-large":  YOLO("yolov8l.pt"),   # Most accurate, slowest
}

for name, model in models_to_test.items():
    times = []
    for _ in range(30):
        start = time.time()
        model(image, device=0, verbose=False)
        end = time.time()
        times.append((end - start) * 1000)

    avg = sum(times) / len(times)
    fps = 1000 / avg
    cameras = fps / 30

    print(f"{name:20s} | {avg:6.1f}ms | {fps:5.1f} fps | {cameras:.0f} cameras")
```

Expected Output (approximate, depends on GPU):

```
Model              |  Time  |   FPS   | Cameras at 30fps
YOLOv8-nano        |   5.2ms | 192 fps |  6 cameras
YOLOv8-small       |   8.1ms | 123 fps |  4 cameras
YOLOv8-medium      |  15.3ms |  65 fps |  2 cameras
YOLOv8-large       |  25.7ms |  39 fps |  1 camera
```

Step 5: Check if It Meets the 33ms Budget

```
yolo_time = 8.1   # From benchmark above
cnn_time = 5.0    # Estimate
lstm_time = 3.0   # Estimate
other_time = 5.0  # Preprocessing, postprocessing

total = yolo_time + cnn_time + lstm_time + other_time
budget = 33.3

print(f"Total pipeline time: {total:.1f}ms")
print(f"Budget: {budget:.1f}ms")
if total < budget:
    print(f"PASS! {budget - total:.1f}ms to spare")
else:
    print(f"FAIL! {total - budget:.1f}ms over budget. Need optimization.")
```

Key Takeaways

- 1. 33ms per frame is the target — anything slower and you fall behind real-time
- 2. Quantization gives easy 2-4x speedup — tiny accuracy loss, huge speed gain
- 3. TensorRT is the gold standard — best performance on NVIDIA GPUs
- 4. Frame skipping is free speed — you do not need every frame to catch cheating
- 5. Pipeline parallelism multiplies throughput — overlap different processing stages
- 6. Always benchmark before deploying — measure, optimize, measure again
- 7. Start with YOLOv8-nano or small — fastest variants, good enough for ExamGuard

Phase 6: Advanced ML — Learning Resources

LSTM and RNN

YouTube Videos

- "LSTM Networks - EXPLAINED!" by CodeEmporium — visual explanation of gates and memory
- "Recurrent Neural Networks (RNN) - Deep Learning w/ Python, TensorFlow & Keras" by sentdex — hands-on coding
- "Illustrated Guide to LSTM's and GRU's" by Michael Phi — best visual walkthrough
- "Understanding LSTM Networks" by The AI Epiphany — step-by-step through the math (beginner-friendly)
- "Sequence Models" by Andrew Ng (YouTube clips from Coursera) — the theory behind it all

Articles

- "Understanding LSTM Networks" by Christopher Olah (colah.github.io) — THE classic explanation, read this first
- "The Unreasonable Effectiveness of Recurrent Neural Networks" by Andrej Karpathy — fun examples

Practice

- Kaggle: "Stock Price Prediction" competitions (time series = sequences)
- PyTorch LSTM tutorial: pytorch.org/tutorials

Autoencoders

YouTube Videos

- "Autoencoders - EXPLAINED" by CodeEmporium — clear visual explanation
- "Building Autoencoders in Keras" by Keras official — step-by-step code
- "Variational Autoencoders" by Arxiv Insights — next level after basic autoencoders
- "Autoencoder Tutorial PyTorch" by Aladdin Persson — full code walkthrough

Practice

- Kaggle: "Credit Card Fraud Detection" dataset — perfect for anomaly detection practice
- MNIST anomaly detection — train on one digit, detect others
- Fashion-MNIST — train on one clothing type, detect others

Anomaly Detection

YouTube Videos

- "Anomaly Detection - Machine Learning" by StatQuest — clear statistical foundation
- "Isolation Forest Algorithm" by Krish Naik — simple explanation with code
- "One-Class SVM" by Normalized Nerd — visual explanation
- "Anomaly Detection in Time Series" by ritvikmath — directly applicable to video

Practice

- Kaggle: "Credit Card Fraud Detection" (most popular anomaly detection dataset)
- Kaggle: "Network Intrusion Detection" (another classic)
- scikit-learn anomaly detection examples: sklearn docs have great tutorials

Reinforcement Learning

YouTube Videos

- "Reinforcement Learning in 3 Hours - Full Course" by Nicholas Renotte — complete beginner course
- "An introduction to Reinforcement Learning" by Arxiv Insights — beautiful visual explanation
- "Reinforcement Learning Course - Full Machine Learning Tutorial" by freeCodeCamp — thorough free course
- "Stable Baselines3 Tutorial" by Nicholas Renotte — the library you will actually use

Courses

- "Reinforcement Learning Specialization" by University of Alberta on Coursera — THE gold standard RL course
- "Deep Reinforcement Learning" by David Silver (DeepMind, YouTube) — legendary lecture series
- "Spinning Up in Deep RL" by OpenAI (spinningup.openai.com) — free, excellent documentation

Practice

- OpenAI Gymnasium environments: CartPole, MountainCar, LunarLander
- Build your own simple environment first
- Stable Baselines3 documentation has many examples

Real-Time Inference and Optimization

YouTube Videos

- "TensorRT Tutorial" by NVIDIA Developer — official guide to GPU optimization
- "ONNX Runtime Tutorial" by Microsoft — convert and speed up models
- "Model Optimization for Edge Deployment" by Edge Impulse — practical optimization
- "PyTorch Model Optimization" by PyTorch official — quantization and pruning

Articles

- NVIDIA TensorRT documentation: developer.nvidia.com/tensorrt
- ONNX Runtime: onnxruntime.ai
- PyTorch Quantization tutorial: pytorch.org/docs/stable/quantization.html

Practice

- Benchmark every model you build (make it a habit)
- Try exporting to ONNX and compare speed
- Experiment with different YOLO model sizes

Recommended Learning Order

Week 1-2: LSTM basics → Next word predictor mini project
Week 3-4: Autoencoders → MNIST anomaly detection mini project
Week 5-6: Anomaly detection → Credit card fraud mini project
Week 7-8: Reinforcement learning → CartPole mini project
Week 9-10: Real-time inference → YOLO benchmarking mini project

Tools to Install

```
# Core ML
pip install torch torchvision
pip install ultralytics        # YOLO

# RL
pip install stable-baselines3
pip install gymnasium

# Optimization
pip install onnx onnxruntime    # ONNX export and runtime
# TensorRT: Follow NVIDIA's install guide for your GPU

# Anomaly detection
pip install scikit-learn        # Isolation Forest, One-Class SVM

# Data
pip install pandas numpy matplotlib seaborn
```

Key Kaggle Competitions for Practice

- 1. Credit Card Fraud Detection — anomaly detection
- 2. IEEE Fraud Detection — advanced anomaly detection
- 3. Stock Market Prediction — LSTM/time series
- 4. Disaster Tweets — NLP sequences (same LSTM skills)
- 5. Any object detection competition — real-time inference practice